Exp 6

Aim :- Creating Smart Contracts and performing transactions using Solidity and Remix IDE.

Theory :-

1. Core Theory of Smart Contracts

What is a Smart Contract?

A smart contract is a self-executing program stored on a blockchain.
It automatically enforces rules and executes actions when predefined conditions are met.

Key characteristics:

- Immutable once deployed — code cannot be changed.
- Distributed — every node stores and verifies the contract.
- Deterministic — same input always produces the same output.
- Transparent — code and state are visible on-chain.

Smart contracts were conceptualized by Nick Szabo (1994) and became mainstream with Ethereum, which introduced a programmable blockchain.

---

2. Solidity: The Language of Ethereum Smart Contracts

Solidity is:

- Contract-oriented (everything revolves around contracts)
- Statically typed
- Compiled into EVM bytecode
- Influenced by JavaScript, C++, and Python

Key Solidity Concepts

Concept	Meaning
pragma	Specifies compiler version
state variables	Stored permanently on blockchain
functions	Define behavior (public, private, view, pure, payable)
events	Emit logs for off-chain listeners

modifiers	Reusable access-control logic
constructor	Runs once during deployment
mappings/structs	Data structures for storage
msg.sender / msg.value	Transaction metadata

---

3. Remix IDE: The Browser-Based Ethereum Development Environment

Remix is a web IDE that allows you to:
- Write Solidity code
- Compile contracts
- Deploy to a virtual blockchain (JavaScript VM)
- Deploy to testnets/mainnet via MetaMask
- Interact with deployed contracts
- Debug transactions

It removes the need for local setup and is ideal for beginners and rapid prototyping.

---

4. The Smart Contract Development Lifecycle

Step 1: Writing the Contract
You create a .sol file and define your contract.

Step 2: Compiling the Contract

Using Remix:
- Open Solidity Compiler
- Select version matching pragma
- Click Compile

Remix checks:
- Syntax errors
- Type mismatches
- Security warnings

Step 3: Deploying the Contract

Using Deploy & Run Transactions panel:
- Choose an environment (JavaScript VM, Injected Provider, etc.)

- Select contract
- Click Deploy

Deployment creates:

- A transaction that stores the contract bytecode on-chain
- A contract address where the contract lives permanently

---

5. Performing Transactions with Smart Contracts

Once deployed, you can interact with the contract.

Types of Transactions

Type	Description
Call (free)	Read-only, no gas, doesn't change blockchain state
Transaction (costs gas)	Writes data, triggers state changes

How Remix Handles Transactions

- Each function appears as a button in the UI.
- Calling a function sends a transaction to the contract.
- Remix shows:
    - Gas used
    - Logs/events
    - Return values
    - State changes

---

6. Payable Functions & Ether Transfers

To send Ether:

```
function deposit() public payable { }
```

To withdraw:

```
function withdraw(uint amount) public {
    payable(msg.sender).transfer(amount);
}
```

Remix allows you to specify value (in wei/ether) before calling payable functions.

---

7. Testing & Debugging in Remix

Remix includes:

- Debugger (step through transactions)
- Console logs
- Gas profiler
- Static analysis tools

These help catch:

- Reentrancy vulnerabilities
- Integer overflows
- Uninitialized storage
- Access control issues

---

8. Why Remix IDE Matters

According to Startupik, Remix lowers the barrier to entry by providing a browser-based environment where developers can write, compile, test, and deploy without complex setup. This makes it ideal for:

- Students
- Hackathon participants
- Blockchain beginners
- Rapid prototyping

---

9. Summary Table

Topic	Key Idea
Smart Contracts	Self-executing blockchain programs
Solidity	Main language for Ethereum contracts
Remix IDE	Browser-based IDE for writing & deploying contracts
Transactions	Calls (free) vs. state-changing (gas)
Deployment	Stores contract on blockchain with unique address
Ether Transfers	Use payable functions
Debugging	Built-in tools for analysis and testing

Code :-

AcademicRecords.sol

```
// SPDX-License-Identifier: MIT pragma solidity 0.8.19; contract AcademicRecords {

struct Student { string name; string course; uint marks; string ipfsHash; bool exists;
}
mapping(string => Student) public students; function addStudent(
string memory _id, string memory _name, string memory _course, uint _marks,
string memory _ipfsHash
) public {
students[_id] = Student(_name, _course, _marks, _ipfsHash, true);
}

function getStudent(string memory _id) public view
returns (string memory, string memory, uint, string memory)
{
require(students[_id].exists, "Student not found"); Student memory s = students[_id];
return (s.name, s.course, s.marks, s.ipfsHash);
}
}
```

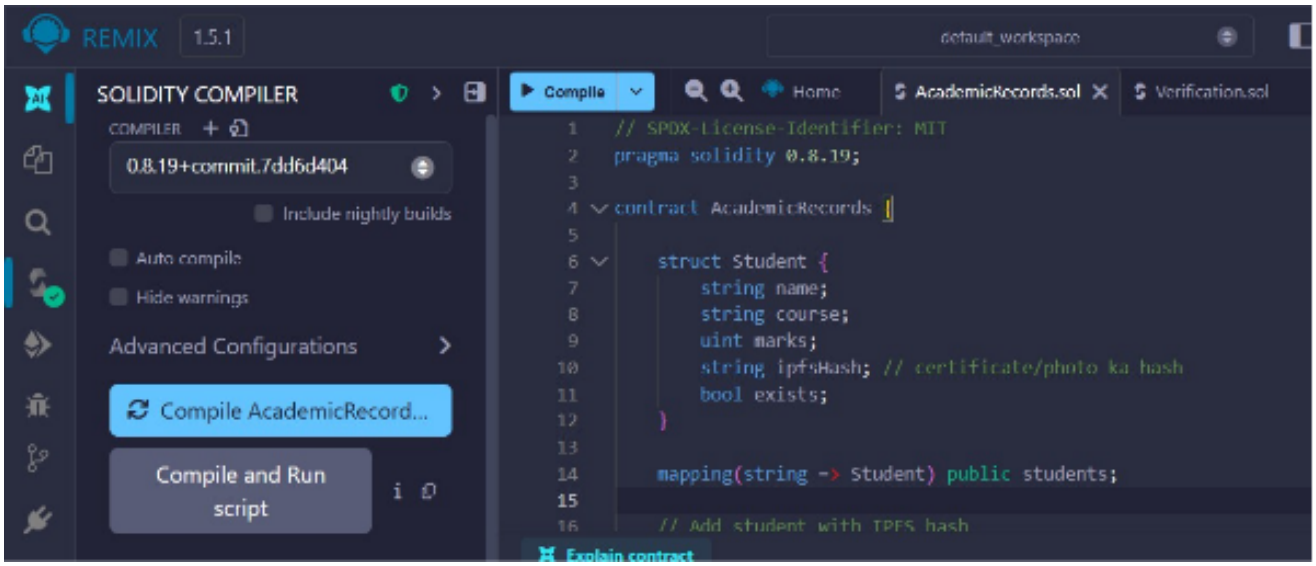
Verification.sol

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.0;

contract Verification {

mapping(string => bool) public verified;

function verifyCertificate(string memory _id) public { verified[_id] = true;
}

function checkVerification(string memory _id) public view
returns (bool)
{
return verified[_id];
}
}
```

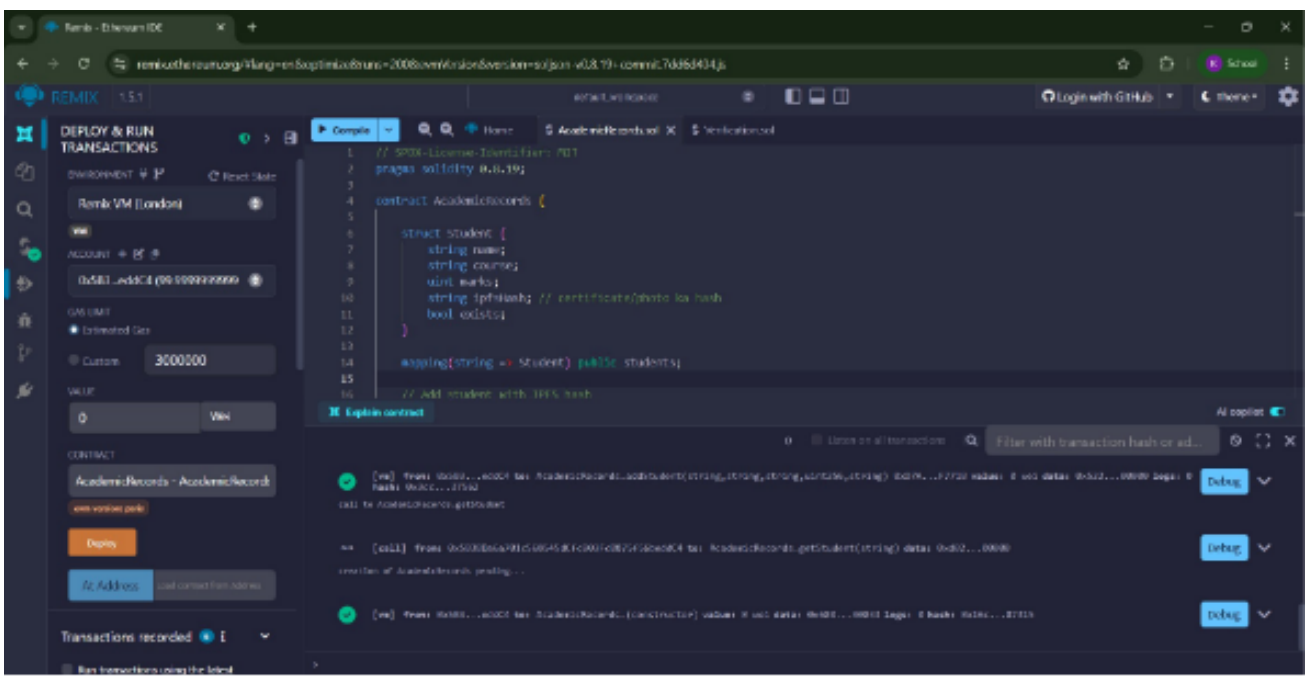
D20A 59 Jai Talreja Blockchain Lab

Screenshots :-

Compiling code :

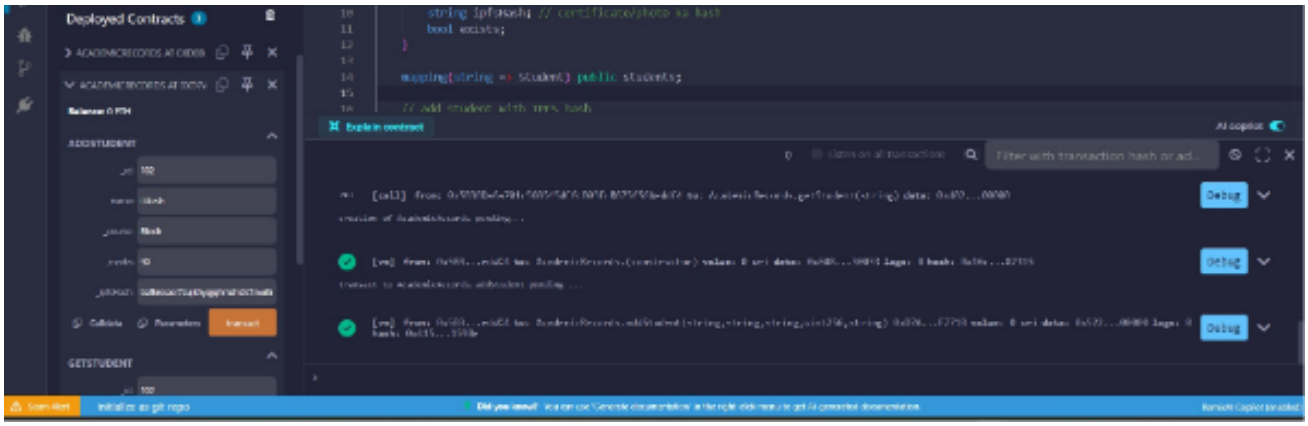


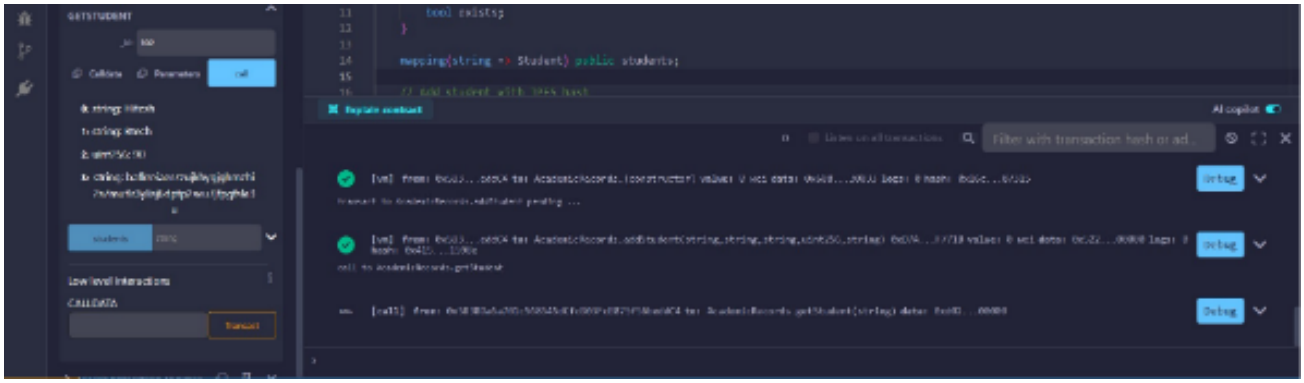
Deploying contract :



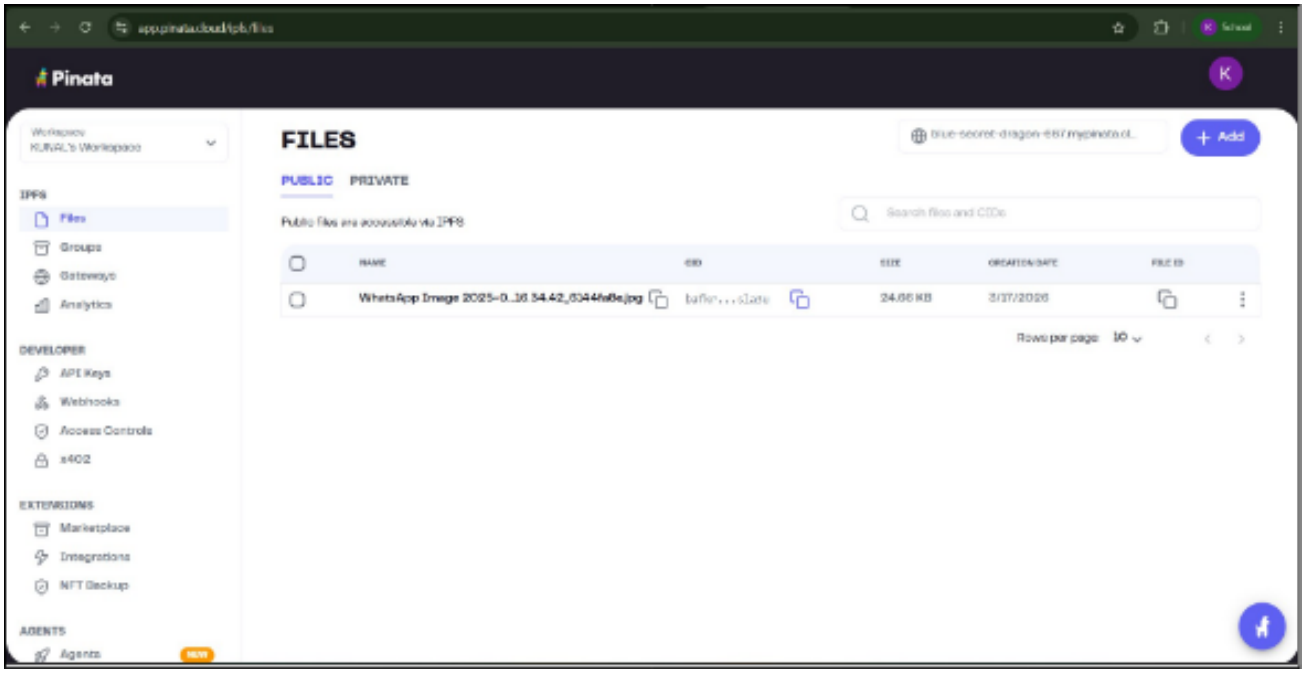
D20A 59 Jai Talreja Blockchain Lab

Contract execution :





Integration :



Conclusion :-

This experiment helped in understanding the practical implementation of smart contracts using Solidity in Remix IDE. A smart contract was successfully designed to store and retrieve student academic records on the blockchain, ensuring data security and immutability. The system was further enhanced by integrating IPFS to store certificate images, with only the hash stored on the blockchain. This approach makes the system efficient, secure, and resistant to data tampering. Overall, the experiment demonstrated how blockchain technology can be effectively used for reliable academic credential verification.